

6FLAS: A Lightweight Security Protection Mechanism for LEO Satellite Networks Using IPv6 Flow Label and SRv6

Anonymous Authors

Abstract—Low Earth orbit (LEO) satellite constellations are becoming a backbone of 6G non-terrestrial networks, yet their open inter-satellite links (ISLs) are exposed to eavesdropping, injection, and replay, while on-board routers offer too little compute and memory to run conventional per-flow security such as IPsec or DTLS. We present 6FLAS, a lightweight, stateless security protection mechanism that moves security state off the satellite and into the packet itself. 6FLAS repurposes the 20-bit IPv6 Flow Label as a connectionless flow authenticator and binds it to an SRv6 security segment, so each on-board router verifies a single keyed message authentication code (MAC) over the routing header using only a small per-direction replay window rather than a per-flow security association table. We formalize the LEO security problem on a resource-constrained directed graph, give the 6FLAS packet format, per-hop verification algorithm, and a security argument against eavesdropping, forgery, man-in-the-middle, replay, and stateless-denial-of-service attacks. We evaluate 6FLAS with a reproducible analytical processing model and an event-driven latency model of a 1584-satellite Walker-delta constellation. 6FLAS reduces per-hop security processing time by about $48\times$ versus IPsec ESP and $5.9\times$ versus an SRv6 HMAC baseline, cuts per-packet byte overhead by 79%, and shrinks per-flow on-board state by 95%, while keeping the end-to-end latency within propagation-dominated bounds. These results indicate that packet-carried, topology-aware security is a practical fit for the resource envelope of LEO on-board routers.

Index Terms—LEO satellite networks, IPv6 Flow Label, SRv6, lightweight authentication, network security, non-terrestrial networks.

I. INTRODUCTION

LOW Earth orbit (LEO) satellite constellations have moved from proposal to deployment: mega-constellations now place thousands of satellites at altitudes of a few hundred kilometers, and standards bodies position them as a backbone of 6G non-terrestrial networks (NTN), delivering low-latency global connectivity where terrestrial infrastructure is absent [1]–[3]. Unlike bent-pipe relays, modern constellations forward traffic in space over inter-satellite links (ISLs), so a packet may traverse many on-board IP routers before reaching a ground gateway [4], [5].

This in-orbit forwarding is exactly what makes security hard. ISLs are broadcast over open free-space channels, and the published, near-deterministic orbital geometry lets an adversary with a suitable terminal predict when and where a link will exist, then eavesdrop, inject, or replay traffic [6]–[9]. At the same time, an on-board router runs on a radiation-hardened

processor with a fraction of the compute and memory of a terrestrial core router, and it cannot be patched or rebooted on demand [10]. The result is a sharp mismatch: the threat surface grows with every added hop, while the per-hop security budget is tiny.

Conventional IP security was designed for endpoints with ample resources and stable associations. IPsec ESP and DTLS protect a flow by maintaining a per-flow security association (SA) holding keys, selectors, counters, and a replay window, and by encrypting and authenticating every packet with heavyweight primitives [11]–[13]. This model assumes endpoints with sufficient memory and relatively stable associations; on a constellation carrying 10^5 – 10^6 concurrent flows, the aggregate SA state and per-packet crypto exceed the on-board budget, and the frequent ISL handovers caused by orbital motion force costly SA re-establishment, with measurement and modeling studies of satellite IPsec reporting significant handover and renegotiation costs [6].

Protocols closer to the space and routing layers help but do not close the gap. At the link layer, the CCSDS Space Data Link Security protocol authenticates and encrypts a single space link [14]; it is effective hop-by-hop but provides neither end-to-end protection across an in-orbit IP path nor any use of the network-layer routing information that LEO IP forwarding already carries. Segment Routing over IPv6 (SRv6) brings programmable source routing to the satellite data plane and is attractive for LEO because a ground controller can pin a path through a predictable constellation [15]–[18]; the SRH defines an HMAC TLV to authenticate the segment list and recent work proposes richer security SIDs [16], [19], [20], but the HMAC TLV authenticates the whole segment list with a full SHA-256 digest that is expensive to verify on every hop, and existing SRv6 security designs still assume per-flow or per-policy state at verifying nodes [20], [21].

Lightweight cryptography supplies fast building blocks but not the missing systems design. A large body of work, including the NIST lightweight-cryptography standardization and software-friendly keyed PRFs such as SipHash, offers primitives suited to constrained hardware [22]–[25], yet these do not by themselves answer where to keep security state or how to verify packets at line rate on an on-board router. Finally, the 20-bit IPv6 Flow Label is set by the source, immutable in transit, and today used only for ECMP load balancing and QoS classification [26], [27]; to our knowledge it has not been developed as a network-layer security authenticator with a defined derivation and verification rule, leaving its potential as

a security primitive unexploited.

We argue that the right move for LEO is to stop storing security state on the satellite and instead carry it in the packet, while exploiting the structured, predictable LEO topology to make per-hop checks trivial. We realize this idea in **6FLAS** (IPv6 Flow Label Authenticated Segments), a lightweight security mechanism that (i) repurposes the IPv6 Flow Label as a connectionless flow authenticator derived from a pre-shared domain key, and (ii) binds it to an SRv6 security segment so that every on-board router verifies a single short keyed MAC over the routing header, using only a small per-direction replay window instead of a per-flow SA table. Because the Flow Label is set by the source and is immutable in transit by design [26], it is a natural carrier for an unforgeable, end-to-end flow identity. Because LEO routers forward over a small number of fixed grid directions, the egress check that normally requires a routing lookup collapses to a constant-time comparison.

This paper makes four contributions, each addressing a gap above.

- **Packet-carried stateless security model for LEO.** We formalize LEO security on a resource-constrained directed graph and show how moving security state from on-board storage into the packet meets the data-origin authentication, integrity, anti-replay, and access-control requirements without per-flow SAs (Section III).
- **Flow-Label-based stateless flow authenticator.** We define a flow as a five-tuple plus a security-domain identifier, derive the 20-bit Flow Label from a domain key, and give a stateless verification rule that needs no per-flow table walk (Section III-B).
- **SRv6 Security SID integration.** We integrate the authenticator with SRv6 through a Security SID (Locator | Function | Arguments) that encodes a path security policy and specializes per-hop checking to the six fixed LEO forwarding directions, while remaining backward compatible with standard SRv6 (Section III-C).
- **Security argument and reproducible evaluation.** We analyze resilience to eavesdropping, forgery, man-in-the-middle, replay, and stateless DoS, and quantify cost with an explicit processing model and an event-driven latency model of a 1584-satellite constellation (Sections IV and V).

On a 1 GHz space-grade core, 6FLAS verifies a packet in about 306 ns per hop, roughly $48\times$ faster than IPsec ESP and $5.9\times$ faster than an SRv6 HMAC baseline; it adds only 8 bytes per packet (79% less than IPsec ESP) and keeps just 12 bytes of on-board state per flow (95% less than IPsec ESP). Under a 10^6 -flow load its single-core forwarding throughput is about $48\times$ that of IPsec ESP, and its end-to-end latency stays within the propagation-dominated regime of the constellation.

The remainder of this paper is organized as follows. Section II gives the LEO, Flow Label, and SRv6 background, the threat model, and the problem formulation. Section III presents 6FLAS. Section IV analyzes its security, and Section V reports the evaluation. Section VI concludes.

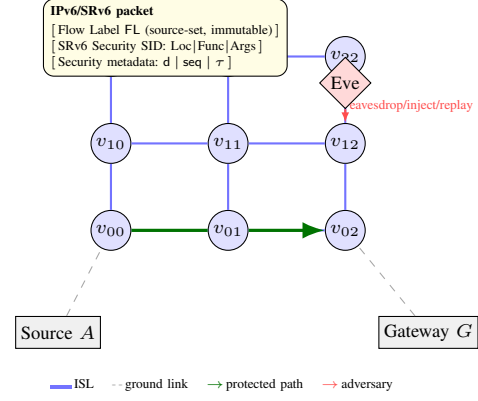


Fig. 1: IPv6-enabled LEO network model. Each satellite is an IPv6/SRv6 on-board router; intra- and inter-plane ISLs form a regular +Grid, and ground links attach sources and gateways. Because every link is an open free-space channel, an adversary (Eve) can eavesdrop, inject, or replay on any ISL or ground link (Section II-C).

II. IPV6-ENABLED LEO NETWORK MODEL AND PROBLEM FORMULATION

This section establishes the network model and problem formulation. We first describe the LEO constellation as a network of IPv6/SRv6 on-board routers and the topological regularities that constrain the design space, then review the two IPv6 features central to lightweight per-hop security: the Flow Label and SRv6. We next state the adversary and the security and design requirements, and finally cast the whole problem as a feasibility condition on a resource-constrained graph. Figure 1 previews the model: an IPv6/SRv6 data plane carried in orbit, the open links an adversary can reach, and the packet fields available for protection.

A. LEO Satellite Network Characteristics

We consider a Walker-delta LEO constellation: P orbital planes each carrying S satellites at altitude h of a few hundred kilometers (we use the Starlink-like shell $h = 550$ km, $P = 72$, $S = 22$, $N = PS = 1584$ in Section V). Each satellite is an on-board IP router connected to its neighbors by ISLs. In the common +Grid pattern, a satellite maintains four ISLs: two intra-plane links to the satellites ahead and behind, and two inter-plane links to satellites in adjacent planes; together with the up/down ground-link direction, this yields a small, fixed set of forwarding directions per node.

Two properties shape the design space. First, the topology is *dynamic but predictable*: satellites move continuously, ISLs are made and broken on handover, yet positions and contact windows follow published ephemerides and can be computed in advance [4], [8], [28], [29]. Second, on-board resources are *severely constrained*: a radiation-hardened processor offers far less compute, memory, and energy than a terrestrial router, and software updates are slow and risky [10], [30]. A security mechanism for LEO must therefore have low per-packet processing cost, low per-flow memory, and must tolerate frequent topology changes without expensive re-keying.

B. IPv6 Flow Label and SRv6 Primer

Two features already present in every IPv6/SRv6 packet are central to lightweight per-hop security: the 20-bit Flow Label, which the source sets and routers must not modify, and the Segment Routing Header, which lets a controller express and enforce an explicit forwarding path. The following paragraphs review each in turn.

a) *IPv6 Flow Label*.: The IPv6 header carries a 20-bit Flow Label field. By RFC 6437 [26] the label is set by the source node, should be uniform across a flow's packets, and *must not be modified by routers in transit*; routers use it as an opaque key for ECMP and QoS without interpreting its value [27]. The field thus provides up to $2^{20} = 1,048,576$ distinct values that travel unmodified end-to-end. These two properties, source-set and immutable, are precisely what a stateless flow authenticator needs.

b) *SRv6*.: Segment Routing over IPv6 expresses an explicit path as an ordered list of 128-bit SIDs carried in the Segment Routing Header (SRH) [15], [16], [19]. A SID follows a `Locator|Function|Arguments` structure [17]: the Locator routes the packet toward the SID's owner, the Function selects a behavior at that node (e.g., End, End.X forwarding to a specific link), and the Arguments parameterize it. A node processing the SRH advances the Segments Left pointer and applies the behavior of the current SID. SRv6 lets a source or controller program an end-to-end path through the constellation [31], [32], and the SRH also defines an optional HMAC TLV to authenticate the segment list [16], [20].

C. Threat Model and Security Requirements

We adopt a Dolev–Yao–style network adversary adapted to LEO. The adversary can (i) *eavesdrop* on any ISL or ground link, since the orbital channel is open and the geometry is predictable; (ii) *inject* arbitrary packets, including spoofed source addresses and forged headers; and (iii) *replay* previously observed packets, possibly from a different vantage point. We assume the adversary *cannot* physically tamper with on-board hardware or extract keys from a satellite's secure element, and cannot break standard cryptographic primitives. Satellites within a security domain share keys provisioned out-of-band before launch or via a separate key-management channel; key management is orthogonal to this work and we treat domain keys as given.

The mechanism must provide: **(R1) data-origin authentication**, a router can confirm a packet originates from an authorized source in its domain; **(R2) integrity**, tampering with the protected header is detected; **(R3) anti-replay**, replayed packets are rejected; and **(R4) access control**, only packets bound to an authorized path/policy are forwarded. Confidentiality of the payload is explicitly *out of scope*; 6FLAS protects the routing and authentication metadata and composes with payload encryption where needed (Section V-E).

In addition, the mechanism must have **(D1) low per-hop processing overhead**, **(D2) low per-flow on-board storage**, and **(D3) robustness to LEO topology dynamics** (handovers without per-flow re-keying).

D. Problem Formulation

We model the constellation at a routing epoch as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is the set of N on-board routers and $(u, v) \in \mathcal{E}$ is an active ISL or ground link. Each node $v \in \mathcal{V}$ has a compute budget C_v (cycles available per packet at line rate) and a memory budget M_v (bytes for security state). A flow f is a stream of packets from an authorized source traversing a path $\pi_f = (v_0, v_1, \dots, v_\ell)$ in $\mathcal{G}$. Let $\mathcal{F}$ be the set of concurrent flows, with $|\mathcal{F}|$ up to 10^6 .

A per-hop security mechanism $\mathcal{M}$ is defined by a verification cost $c_{\mathcal{M}}$ (cycles per packet per hop) and a per-flow state cost $m_{\mathcal{M}}$ (bytes per flow per node). Feasibility under Section II-A requires, for every node v on the busiest path,

$$c_{\mathcal{M}} \leq C_v \quad \text{and} \quad \sum_{f: v \in \pi_f} m_{\mathcal{M}} \leq M_v. \quad (1)$$

The goal is to design $\mathcal{M}$ that satisfies the security requirements R1–R4 while minimizing $c_{\mathcal{M}}$ and, crucially, the per-flow term $m_{\mathcal{M}}$, since the state cost in Equation (1) scales with the number of flows crossing a node. Stateful mechanisms make $m_{\mathcal{M}} = \Theta(1)$ *per flow* (a full SA), so the left-hand sum grows linearly in the flow count; the ideal mechanism targets $m_{\mathcal{M}} \rightarrow 0$ per flow by carrying the security context in the packet, leaving only a bounded per-direction replay window at each node, independent of $|\mathcal{F}|$.

III. 6FLAS: LIGHTWEIGHT SECURITY MECHANISM

We now present 6FLAS. We begin with the two design principles that motivate the mechanism, then describe its two cooperating carriers: a Flow-Label-based stateless flow authenticator and an SRv6-integrated security segment that drives constant-time per-hop verification. We close with domain key management and an end-to-end walk-through of the data path. Throughout, the design keeps a single invariant in view, that the only mutable on-board state is a small per-direction replay window, never anything per flow.

A. Design Philosophy

6FLAS rests on two ideas. First, *carry security state in the packet, not on the satellite*. A stateful protocol stores a security association per flow at every protected node; 6FLAS instead derives a compact authenticator from a domain key and embeds it in fields the packet already carries, so a verifying router needs no per-flow record. This drives the per-flow state term $m_{\mathcal{M}}$ in Equation (1) toward zero and makes the mechanism indifferent to ISL handovers: when a flow is rerouted, no SA migrates because none exists. Second, *exploit the structured LEO topology*. Because each router forwards over a small fixed set of grid directions (Section II-A), the egress access-control check, which on a general router needs a routing/policy lookup, becomes a constant-time comparison against the SID-encoded expected direction.

The two carriers are complementary. The immutable, source-set Flow Label (Section II-B) supplies an end-to-end, unforgeable flow identity that is the same at every hop; the SRv6 Security SID supplies the per-hop path policy and the bytes over which integrity is checked. Figure 2 shows the resulting packet and the per-hop pipeline.

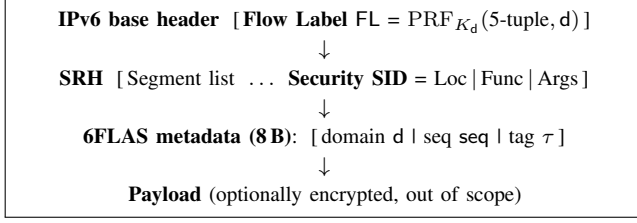


Fig. 2: 6FLAS packet structure. The Flow Label carries a stateless flow authenticator; the SRv6 Security SID encodes the per-hop path policy; an 8-byte metadata block carries the domain id, sequence number, and truncated MAC tag. Each on-board router verifies the tag over the shaded routing span and checks the replay window, then forwards.

B. Flow-Label-Based Stream Authentication

Definition 1 (Authenticated flow). A 6FLAS flow is the tuple $f = (5tuple, d)$ where $5tuple$ is the IPv6 five-tuple (source/destination address, source/destination port, next-header) and d is a security-domain identifier shared by the authorized routers serving the flow.

The source node sets the Flow Label deterministically from the flow and the domain key K_d :

$$FL = \text{PRF}_{K_d}(5tuple \parallel d) \bmod 2^{20}, \quad (2)$$

where PRF is a keyed pseudorandom function (we use a SipHash-class keyed PRF [24]). By RFC 6437 the label is then immutable in transit [26], so FL is a stable, source-bound token that every hop observes identically. Because K_d is secret, an off-path adversary cannot compute the label that a given five-tuple should carry, and an on-path adversary that copies FL cannot reuse it for a different five-tuple without detection (the per-packet MAC below binds them together). The 20-bit space admits up to $2^{20} \approx 1.05\text{M}$ concurrent labels per domain (Section V-B discusses collision behavior).

Each packet carries an 8-byte 6FLAS metadata block: a 1-byte domain id d , a 3-byte monotonically increasing sequence number seq , and a 4-byte truncated MAC tag

$$\tau = \text{MAC}_{K_d}(FL \parallel SID \parallel d \parallel seq)_{32}, \quad (3)$$

computed over the immutable Flow Label, the active Security SID, the domain id, and the sequence number. A verifying router recomputes Equation (3) from fields already in the packet plus the domain key it holds; no per-flow state is consulted. The tag binds the flow identity (FL), the path policy (SID), and the freshness (seq) together, so altering any of them invalidates the tag (R1, R2). Section IV bounds the forgery probability of the 32-bit truncation.

C. SRv6 Security Segment and Per-Hop Verification

6FLAS encodes its per-hop policy in an SRv6 Security SID that follows the standard `Locator|Function|Arguments` layout [17], so legacy SRv6 nodes that do not understand the 6FLAS function still route on the Locator and skip the behavior, preserving backward compatibility.

Algorithm 1 6FLAS per-hop verification at router v

Require: packet p ; domain keys $\{K_d\}$; replay windows $W_v[\cdot]$

```

1: (FL, SID, d, seq,  $\tau$ )  $\leftarrow$  PARSE( $p$ )
2: if  $d \notin$  authorized domains at  $v$  then
3:   drop
4: end if
5:  $w \leftarrow W_v[\text{hash}(FL) \% |W_v|]$   $\triangleright O(1)$  window select
6: if  $seq \leq w.\text{base}$  or  $w.\text{seen}(seq)$  then
7:   drop  $\triangleright$  replayed / stale (R3)
8: end if
9:  $\tau' \leftarrow \text{MAC}_{K_d}(FL \parallel SID \parallel d \parallel seq)_{32}$ 
10: if  $\tau' \neq \tau$  then
11:   drop  $\triangleright$  forged / tampered (R1,R2)
12: end if
13:  $\text{dir} \leftarrow \text{EGRESS}(\text{SID.Args})$ 
14: if  $\text{dir} \notin$  authorized directions at  $v$  then
15:   drop  $\triangleright$  policy violation (R4)
16: end if
17:  $w.\text{ADVANCE}(seq)$ ; APPLYSRV6( $p$ ); forward
  
```

- **Locator** (64 b): routes the packet toward the next 6FLAS node, exactly as in standard SRv6.
- **Function** (16 b): a code point `End.SEC` that triggers 6FLAS verification before the usual `End/End.X` forwarding.
- **Arguments** (48 b): the path security policy, encoding the expected ingress direction, the authorized egress direction (one of the six fixed LEO grid directions), and a domain/policy tag matched against d .

Encoding the expected egress direction in the SID is what lets a router enforce access control (R4) without a routing-table or policy lookup: the controller that programs the path writes the intended next direction into the SID at source time, and each hop simply checks that its actual forwarding direction matches.

a) Per-hop verification: Algorithm 1 gives the per-hop procedure. After parsing the IPv6 base header and the active SID, the router (1) hashes the Flow Label into a small per-direction replay-window table to locate the relevant window in $O(1)$, (2) checks the sequence number against the sliding window for anti-replay (R3), (3) recomputes the MAC of Equation (3) and compares it in constant time to the carried tag (R1, R2), and (4) checks that the SID-encoded egress direction is one of the node's authorized directions before forwarding (R4). Every step uses only fields in the packet and the domain key; there is no per-flow SA lookup.

b) Topology-aware optimization and statelessness.: A general router enforcing an egress policy would consult a forwarding or policy table whose size grows with the number of destinations. In a +Grid LEO node the authorized egress set has at most six members (north/south intra-plane, east/west inter-plane, up/down ground), so line 11 of Algorithm 1 is a comparison against a six-element fixed set, i.e., constant time and constant space, independent of $|\mathcal{F}|$. Likewise the replay window table W_v is sized per direction, not per flow, so its memory is $O(1)$ in the flow count. These specializations are

what make 6FLAS satisfy the feasibility condition Equation (1) at 10^6 flows where stateful schemes do not. The only mutable state a 6FLAS router keeps is the bounded per-direction replay window W_v , independent of the number of flows: many flows hash into the same window, and the window only tracks recently seen sequence numbers within a time horizon (Section IV). Thus $m_{\mathcal{M}}$ per flow is effectively zero, and a handover that reroutes a flow requires no state transfer, directly satisfying D2 and D3.

D. Domain Key Management

6FLAS verification needs each authorized router to hold the domain key K_d ; how that key is distributed and rotated is a deployment choice, but the stateless design makes it light. We sketch a workable scheme and note its properties; a full key-management protocol is out of scope (Section V-E).

A constellation operator runs a ground key-management service that owns a per-domain master key. Each satellite is provisioned, at manufacture or via a secured telecommand channel, with the keys of the domains it serves; because a domain typically maps to an administrative slice of the constellation rather than to a flow, the number of keys a node holds is small and static, in contrast to the per-flow keying that stateful schemes require. Keys are rotated on an epoch schedule: the 1-byte domain id d doubles as a key-epoch selector, the operator pushes $K_d^{(e+1)}$ to all nodes before epoch $e+1$ begins, sources start stamping packets with the new epoch, and verifiers accept both the current and the immediately previous epoch during a short overlap to absorb in-flight packets. Rotation is a control-plane push that is independent of the number of flows and does not interrupt forwarding. If a satellite is suspected compromised, the operator rotates the affected domain key and re-provisions the remaining nodes, which immediately invalidates any traffic the adversary could mint with the old key; because 6FLAS holds no per-flow secrets, revocation granularity is the domain — coarse, but cheap and consistent with the operator-controlled trust boundary of an intra-constellation deployment.

To make the data path concrete, consider a packet from source A to gateway G routed through three on-board nodes $v_1 \rightarrow v_2 \rightarrow v_3$ in domain $d=7$. At the source, A computes $FL = \text{PRF}_{K_7}(\text{5tuple} \parallel 7) \bmod 2^{20}$, writes it into the IPv6 Flow Label, pushes an SRH whose active Security SID at each hop encodes the intended egress direction (e.g., east, north, down), sets seq from its per-flow counter, and computes the 32-bit tag $\tau = \text{MAC}_{K_7}(FL \parallel SID \parallel 7 \parallel seq)_{32}$. At v_1 , the router reads $d=7$ (authorized), selects the replay window by $\text{hash}(FL)$, checks seq , recomputes the tag over the same span and matches it, confirms its egress is east, advances the window, and forwards. Nodes v_2 and v_3 repeat the identical $O(1)$ check against their own authorized directions. No node ever created, looked up, or migrated a per-flow record; if an ISL break reroutes the flow through a different v'_2 , that node verifies the next packet with K_7 exactly as v_2 would have, with no re-keying.

IV. SECURITY ANALYSIS

We analyze 6FLAS against the threat model of Section II-C. Throughout, let K_d be the secret domain key, assume the keyed PRF/MAC is a secure pseudorandom function, and recall that the Flow Label is immutable in transit (Section II-B).

A. Resilience to Known Attacks

We examine 6FLAS against the four attack classes that follow from the Section II-C threat model: forgery and tampering, replay, man-in-the-middle manipulation, and stateless denial of service.

a) *Forgery and tampering (R1, R2).*: A packet is accepted only if the carried tag equals $\tau = \text{MAC}_{K_d}(FL \parallel SID \parallel d \parallel seq)_{32}$ (Equation (3), Algorithm 1). An adversary without K_d who modifies any covered field, the flow identity FL, the path policy SID, the domain id, or the sequence number, must produce a matching tag.

Proposition 2 (Forgery bound). *Against an adversary making q verification queries and lacking K_d , the probability that any single forged or tampered packet is accepted is at most $2^{-32} + \epsilon_{\text{PRF}}$, where ϵ_{PRF} is the PRF-distinguishing advantage for the keyed MAC.*

The 2^{-32} term is the chance of guessing the 32-bit truncated tag; a secure MAC contributes only ϵ_{PRF} . We prove this in Section IV-C. Truncating to 32 bits trades tag length for header compactness; Section V-E discusses widening the tag when a stronger bound is required. Because the tag covers SID, an adversary also cannot redirect a packet onto an unauthorized path without invalidating it (R4).

b) *Replay (R3).*: Each accepted packet advances the per-direction sliding window W_v (Algorithm 1, lines 4–6, 12). A packet whose sequence number is below the window base or already marked seen is dropped. The window covers a bounded time horizon Δ ; combined with the loosely synchronized clocks available from the constellation's timing service, packets older than Δ are rejected regardless of window occupancy, so an adversary cannot replay an old packet even after the window slides past it. Replying within Δ fails because the sequence number is already recorded.

c) *Man-in-the-middle.*: An on-path adversary can observe and relay packets but cannot alter the covered fields without breaking Theorem 2, and cannot splice a valid tag from one flow onto another because the tag binds FL (the per-flow identity) to the SID and sequence number. Downgrade to plain SRv6 is detected because a 6FLAS domain expects the `End.SEC` function and a valid tag; a packet lacking them is dropped at the first 6FLAS hop.

d) *Denial of service.*: Stateful protocols are vulnerable to state-exhaustion DoS: an attacker opens many bogus flows to fill the SA table. 6FLAS keeps no per-flow state, so this vector is closed by construction. The first action in Algorithm 1 is a domain-id check, and verification is a single MAC plus an $O(1)$ window probe, so the per-packet work an attacker can induce is bounded and small. A Flow-Label allow-list per domain further lets a node reject traffic outside its assigned label ranges before computing any MAC.

TABLE I: Security properties provided ($\bullet$ = yes, $\circ$ = partial, $-$ = no/not applicable).

Property	IPsec	DTLS	SRv6-HMAC	6FLAS
Data-origin auth. (R1)	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Integrity (R2)	$\bullet$	$\bullet$	$\bullet$	$\bullet$
Anti-replay (R3)	$\bullet$	$\bullet$	$\circ$	$\bullet$
Path access control (R4)	$\circ$	$-$	$\circ$	$\bullet$
Stateless verification	$-$	$-$	$-$	$\bullet$
Handover w/o re-key	$-$	$-$	$\circ$	$\bullet$
Payload confidentiality	$\bullet$	$\bullet$	$-$	$-$

Taken together, the four analyses confirm that 6FLAS satisfies R1–R4 under the threat model. Forgery and tampering are bounded by the MAC truncation (Theorem 2); replay is eliminated by the sliding window; man-in-the-middle manipulation is detected because the tag binds flow identity, path policy, and sequence number jointly; and denial-of-service via state exhaustion is structurally impossible because no per-flow state exists to exhaust. The cost of each check is $O(1)$ in the flow count, consistent with the feasibility condition of Section II-D.

B. Comparative Security Properties

Table I compares the security properties 6FLAS provides against the baselines. 6FLAS matches IPsec/DTLS on authentication, integrity, and anti-replay while uniquely offering stateless verification and topology-aware access control; it deliberately omits payload confidentiality, which it leaves to a composable encryption layer (Section V-E).

C. Formal Security Argument

We now make the authentication guarantee precise by reducing it to the unforgeability of the keyed MAC.

Proof of Theorem 2. Let MAC_K be the keyed MAC of Equation (3) and let $\text{MAC}_K|_{32}$ be its 32-bit truncation. Consider an adversary $\mathcal{A}$ that, after observing honestly generated packets, submits a packet whose covered message $M = (\text{FL} \parallel \text{SID} \parallel \text{d} \parallel \text{seq})$ was never produced by an honest source. (If M equals an honest message, the packet is a replay and is rejected by the sliding window of Algorithm 1, so we may assume M is fresh.) Acceptance requires the carried tag to equal $\text{MAC}_K(M)|_{32}$.

Build a forger $\mathcal{B}$ against MAC_K : $\mathcal{B}$ forwards $\mathcal{A}$'s observation requests to its own tagging oracle to generate honest packets, and outputs $\mathcal{A}$'s fresh pair (M, τ) as a forgery. Because M was never queried, this is a valid forgery attempt. If MAC_K is a secure PRF with distinguishing advantage ϵ_{PRF} , the probability that τ matches the true full tag is at most ϵ_{PRF} beyond random guessing. With 32-bit truncation, an adversary that simply guesses succeeds with probability 2^{-32} . Combining, the per-packet acceptance probability of a fresh forgery is at most $2^{-32} + \epsilon_{\text{PRF}}$. Summing over q verification attempts gives an overall bound of $q(2^{-32} + \epsilon_{\text{PRF}})$, the standard multi-query restatement of the single-packet bound. $\square$

The reduction shows the structure of the guarantee: a successful network adversary $\mathcal{A}$ that produces an accepted

packet it never observed from an honest source is exactly a MAC forger, because the covered fields it submits differ from any honestly produced message (otherwise the packet is a replay, caught by the window). Hence 6FLAS's authentication security reduces to the unforgeability of its keyed MAC, with the 32-bit truncation contributing the 2^{-32} term of Theorem 2.

V. PERFORMANCE EVALUATION AND DISCUSSION

This section quantifies the cost of 6FLAS and contrasts it with conventional mechanisms. We first describe the reproducible two-part model and the baselines, then report results spanning per-hop processing, communication and state overhead, throughput under load, end-to-end latency, handover cost, payload sensitivity, and the tag-width tradeoff. We close by discussing where 6FLAS fits, how it maps to hardware, and its limitations.

A. Evaluation Setup

We evaluate 6FLAS with a reproducible two-part model rather than measurements on flight hardware, which we do not have access to. An *analytical processing model* charges each mechanism the cost of its per-hop primitives in CPU cycles on a 1 GHz space-grade core (1 cycle = 1 ns), using primitive costs drawn from standard embedded software-crypto microbenchmarks; parsing, MAC, encryption, replay, and association-lookup terms are listed explicitly so the model can be audited and re-parameterized. An *event-driven latency model* of the constellation routes random source–destination pairs over a +Grid topology and accumulates free-space propagation, per-hop security processing, and light $M/M/1$ queueing. The constellation is a Starlink-like shell: $h = 550$ km, $P = 72$ planes, $S = 22$ satellites per plane, $N = 1584$ nodes, intra-plane ISL ≈ 1970 km and inter-plane ISL ≈ 604 km [30], [33]. The entire evaluation is regenerated deterministically (seed fixed) by a single script released with the paper.

We compare 6FLAS against four baselines: **Plain SRv6** (routing only, no security; a lower bound on processing), **IPsec ESP** (transport mode, HMAC-SHA256 + AES-128-CBC, per-flow SA), **DTLS 1.2** (record layer, same primitives), and **SRv6-Sec (HMAC)**, which verifies an HMAC-SHA256 over the routing header per hop in the spirit of the SRH HMAC TLV [16]. We report four categories of metrics: **per-hop cost** (processing time, communication overhead, on-board state); **scalability** (forwarding throughput vs. concurrent-flow count); **dynamics** (end-to-end latency and control-plane handover cost); and **sensitivity** (payload-size independence, tag-width tradeoff, and energy).

Table II lists the per-primitive cycle costs used by the processing model, normalized to a 1 GHz core (1 cycle = 1 ns). Costs follow widely reported embedded software-crypto ratios: symmetric MAC/cipher work is charged per processed byte or per 16-byte block, with fixed setup terms for HMAC and AES. The security-relevant covered span for 6FLAS and the SRv6 HMAC baseline is the IPv6 base header (40 B) plus the SRH with one SID (24 B) plus the 6FLAS metadata block (8 B), totaling 72 B. IPsec/DTLS additionally process a 512 B payload, reflecting their payload encryption. All constants and the resulting figures are produced by the released script.

TABLE II: Per-primitive cost constants (cycles) used in the processing model.

Primitive	Cost (cycles)
Parse IPv6 base header	40
Parse SRH + one SID	60
Flow-Label window select	25
Replay sliding-window test	30
SipHash keyed MAC (per byte)	2.1
HMAC-SHA256 setup	900
HMAC-SHA256 (per byte)	11.0
AES-128-CBC (per 16 B block)	220
AES setup	350
IPsec SA / SPD lookup	180
DTLS record bookkeeping	1500

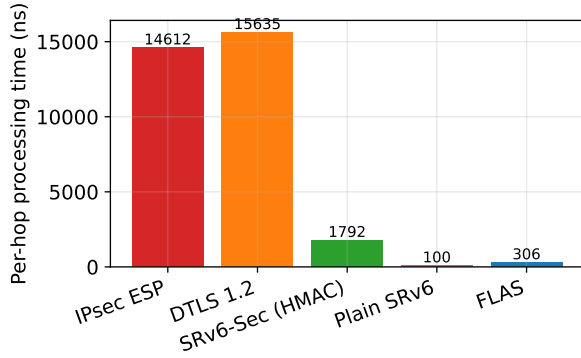


Fig. 3: Per-hop security processing time on a 1 GHz space-grade core. 6FLAS stays close to routing-only Plain SRv6 and is far below the HMAC/encryption baselines. (Lower is better.)

B. Per-Hop Cost and Communication Overhead

Figure 3 shows per-hop processing time. 6FLAS verifies a packet in about 306 ns, dominated by the short keyed MAC over the 72-byte covered span and the $O(1)$ replay-window probe. IPsec ESP costs about 14.6 μ s and DTLS about 15.6 μ s, since both run a full HMAC-SHA256 plus AES-CBC over the payload and pay an SA/record lookup; the SRv6 HMAC baseline costs about 1.79 μ s because of its full SHA-256 digest. 6FLAS is thus about 48 $\times$ faster than IPsec ESP and 5.9 $\times$ faster than the SRv6 HMAC baseline, while remaining within a small constant factor of routing-only Plain SRv6.

In communication overhead (Figure 4), 6FLAS adds only 8 bytes (the metadata block) and reuses the existing Flow Label field at no extra cost, whereas IPsec ESP adds 38 bytes (ESP header, IV, ICV, trailer), DTLS adds 37, and the SRv6 HMAC TLV adds 36 (4-byte TLV header + 32-byte digest). 6FLAS therefore cuts header overhead by about 79% relative to IPsec ESP, which matters on the bandwidth-limited ISL.

C. State, Scalability, and Dynamics

Table III lists per-flow on-board state. 6FLAS keeps 12 bytes per flow (a replay-window pointer), 95% less than IPsec's 256-byte SA and 96% less than DTLS's 320-byte session. Crucially, 6FLAS's mutable state is sized per direction, not per flow, so it does not grow with $|\mathcal{F}|$. Figure 5 sweeps the concurrent-

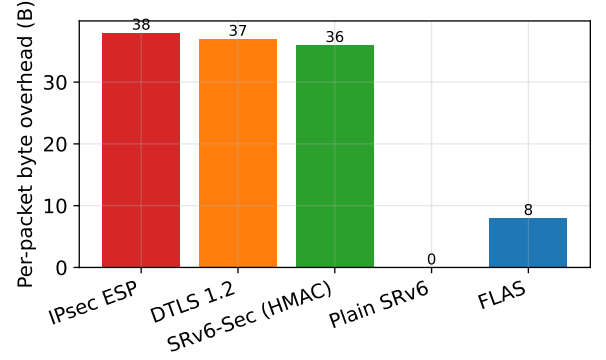


Fig. 4: Per-packet byte overhead added by each mechanism on top of IPv6+SRH. 6FLAS adds only an 8-byte metadata block. (Lower is better.)

TABLE III: Per-flow on-board security state.

Mechanism	State / flow (B)	Reduction vs. IPsec
IPsec ESP	256	—
DTLS 1.2	320	−25% (worse)
SRv6-Sec (HMAC)	96	63%
6FLAS	12	95%

flow count from 10 to 10⁶ and plots single-core forwarding throughput. Stateful baselines degrade as the working set grows (association-lookup cache pressure), whereas 6FLAS and Plain SRv6 are flat. At 10⁶ flows 6FLAS sustains about 3.3 Mpps on one core, roughly 48 $\times$ the throughput of IPsec ESP under the same load.

End-to-end latency and handover cost. Figure 6 shows the latency CDF over the constellation for 6FLAS, IPsec ESP, and Plain SRv6. End-to-end latency is dominated by free-space propagation and queueing across the multi-hop path: 6FLAS's median (≈ 99 ms) and 95th percentile (≈ 189 ms) are statistically indistinguishable from Plain SRv6 (≈ 98 ms median) and slightly better than IPsec ESP, whose larger per-hop processing adds a small but measurable tail. The takeaway is that 6FLAS's security processing is negligible at the path level; its real advantage, shown in Figures 3 and 5, is the per-router compute and state budget, which is what actually constrains an on-board forwarding engine.

The LEO data plane is dominated by churn: orbital motion continually makes and breaks ISLs, so flows are rerouted onto new satellites many times per orbit. A stateful mechanism must migrate or re-establish its per-flow security association on the new node, which means a fresh handshake (an asymmetric operation) and an SA/session install; we charge IPsec a $\sim$ IKEv2-class handshake and DTLS a comparable record-layer handshake, and the SRv6 HMAC baseline a per-flow key re-derivation. 6FLAS migrates *nothing*: the security context travels in the packet, so the next satellite verifies the very next packet with the domain key it already holds.

Figure 7 sweeps the number of flows that must be rerouted within a one-second control window and reports the aggregate control-plane CPU time this imposes on the new node. At

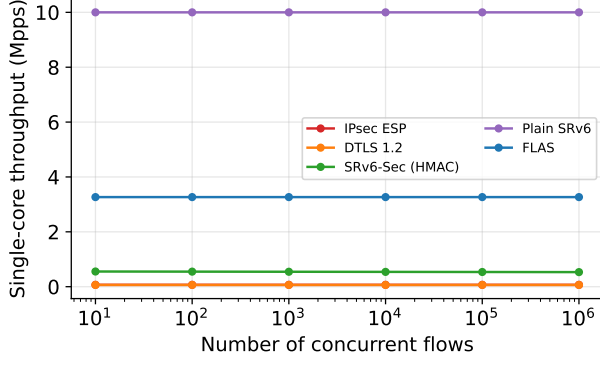


Fig. 5: Single-core forwarding throughput versus number of concurrent flows. 6FLAS (stateless) tracks Plain SRv6 and stays flat to 10^6 flows, while stateful baselines degrade under association-lookup pressure. (Higher is better.)

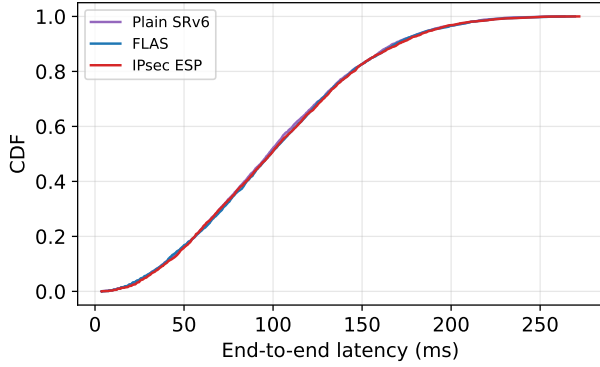


Fig. 6: End-to-end latency CDF over the 1584-satellite constellation. 6FLAS is indistinguishable from routing-only Plain SRv6 because propagation dominates; the security cost does not appear at the path level.

10^4 rerouted flows, IPsec must spend on the order of 35 s of control-plane CPU time and DTLS about 38 s (far beyond a one-second budget, i.e., the node cannot keep up), the SRv6 HMAC baseline about 1.2 s, while 6FLAS spends none. This is the practical consequence of design goal D3: 6FLAS turns a handover from a re-keying event into a non-event, which is exactly the property a churn-dominated constellation needs.

D. Sensitivity Analysis

6FLAS authenticates only the fixed 72-byte routing span (Table II), so its per-hop cost is constant regardless of how large the data packet is. IPsec and DTLS authenticate and encrypt the payload, so their per-hop cost grows linearly with packet size. Figure 8 sweeps the payload from 64 to 1500 bytes: 6FLAS and the SRv6 HMAC baseline are flat (306 ns and $1.79 \mu\text{s}$), whereas IPsec rises from about $4.6 \mu\text{s}$ to $39 \mu\text{s}$ and DTLS similarly. On an ISL carrying full-size data packets, the gap between 6FLAS and payload-processing schemes therefore widens with packet size, reaching roughly $128\times$ at 1500 bytes.

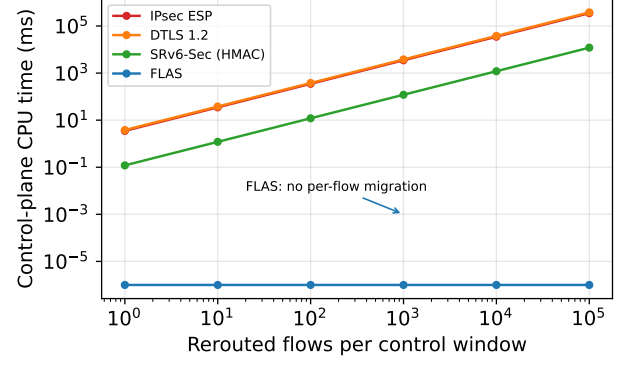


Fig. 7: Control-plane CPU time to re-secure flows after an ISL handover, versus the number of rerouted flows in a one-second window (log-log). Stateful schemes pay a handshake/SA-install per rerouted flow; 6FLAS pays nothing because no per-flow state migrates. (Lower is better.)

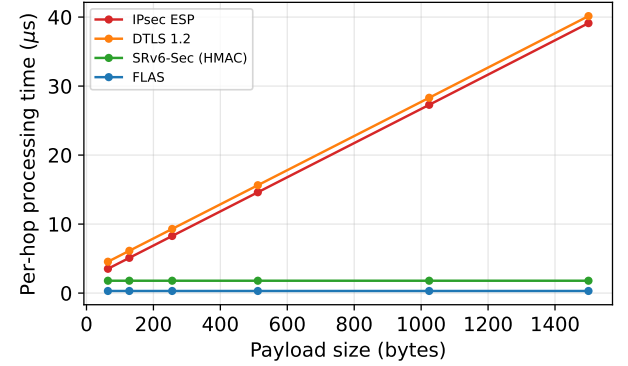


Fig. 8: Per-hop processing time versus payload size. 6FLAS authenticates a fixed routing span, so its cost is flat; payload-processing schemes grow linearly. (Lower is better.)

Authenticating routing metadata rather than payload is what makes 6FLAS's cost insensitive to the traffic it carries.

The truncated tag width t sets both the single-packet forgery bound (2^{-t} , Theorem 2) and the metadata size. Table IV tabulates the tradeoff. Widening the tag from the 32-bit default to 64 bits tightens the forgery bound from 2^{-32} to 2^{-64} at a cost of only 4 extra bytes per packet and 8 ns of extra processing, because the keyed MAC already runs over the routing span and only the tag-comparison span grows. A deployment can thus pick its security margin almost for free; we use 32 bits as a balance between the bound and header compactness, and note that even the 64-bit variant still adds only 12 bytes, less than a third of IPsec's 38.

Energy is a first-order constraint on a solar/battery-limited satellite. Using a representative space-grade figure of 0.5 nJ per cycle, 6FLAS spends about 153 nJ to verify a packet, versus $7.3 \mu\text{J}$ for IPsec ESP and 896 nJ for the SRv6 HMAC baseline, a $48\times$ energy saving over IPsec that compounds over every hop of every packet across the constellation's lifetime. The 20-bit Flow Label gives $2^{20} \approx 1.05\text{M}$ values per security domain;

TABLE IV: Tag-width tradeoff: forgery bound, metadata size, and per-hop cost.

Tag (bits)	Forgery bound	Metadata (B)	Proc. (ns)
16	2^{-16}	6	302
24	2^{-24}	7	304
32	2^{-32}	8	306
48	2^{-48}	10	310
64	2^{-64}	12	315

with 256 domains the effective identifier space is $\approx 2.7 \times 10^8$, comfortably above the 10^6 concurrent-flow target. Within one domain, deriving FL via a keyed PRF (Equation (2)) spreads flows uniformly, so collisions begin to appear as the active-flow count approaches $\sqrt{2^{20}} \approx 1024$ per domain — a collision is not a security failure because the per-packet MAC still binds the full five-tuple, and large deployments avoid it by partitioning across domains.

E. Deployment, Limitations, and Future Work

a) *Deployment fit and implementation.*: 6FLAS targets the regime that breaks conventional IP security: many concurrent flows, tiny per-router budgets, and frequent handovers. Its stateless design removes the dominant cost (per-flow associations) and is robust to the topology churn that forces SA renegotiation in stateful schemes. It is best suited to intra-constellation forwarding within an administrative domain that can provision domain keys. Algorithm 1 maps cleanly to hardware: a keyed-MAC core, a small replay-window SRAM, and a six-entry direction comparator suffice, so a 6FLAS data path can be realized in an FPGA/ASIC pipeline at line rate [34], [35]. On the existing LEO IP/SRv6 data plane, 6FLAS adds the End.SEC function and reuses the standard SRH, so legacy nodes interoperate by routing on the Locator.

b) *Limitations and future work.*: 6FLAS provides authentication, integrity, anti-replay, and path access control, but *not* payload confidentiality; deployments needing secrecy compose 6FLAS with an encryption layer, paying that cost only where required. The anti-replay window relies on loose time synchronization, which LEO constellations already provide but which adds a dependency. The 32-bit tag bounds single-packet forgery at 2^{-32} (Theorem 2); applications needing a stronger bound can widen the tag at a small header cost. Domain-key distribution and revocation operate at domain rather than per-flow granularity (Section III-D); a full key-management protocol with formal guarantees remains an important direction. Future work includes a confidentiality extension that selectively encrypts payloads while preserving stateless verification, a synchronization-free anti-replay variant based on monotonic per-source epochs, and engagement with standards bodies on a Flow-Label security profile and the End.SEC SRv6 behavior.

VI. CONCLUSION

LEO constellations forward IP traffic in orbit across many on-board routers whose compute and memory cannot absorb conventional per-flow security, yet whose open ISLs demand protection. We presented 6FLAS, which resolves this tension

by moving security state out of the satellite and into the packet: it derives a stateless flow authenticator from a domain key and carries it in the immutable IPv6 Flow Label, binds it to an SRv6 Security SID that encodes the per-hop path policy, and verifies each packet with a single short keyed MAC plus a constant-time, topology-aware egress check. The design needs no per-flow security association, so it survives ISL handovers without re-keying and resists state-exhaustion denial of service by construction.

Our analysis shows 6FLAS meets data-origin authentication, integrity, anti-replay, and path access-control requirements, with single-packet forgery bounded by the MAC and the 32-bit tag. A reproducible processing model and an event-driven model of a 1584-satellite constellation show 6FLAS cuts per-hop processing about $48\times$ versus IPsec ESP and $5.9\times$ versus an SRv6 HMAC baseline, reduces per-packet overhead by 79% and per-flow state by 95%, and keeps end-to-end latency within the propagation-dominated regime. These results indicate that lightweight, stateless, packet-carried security is a practical fit for resource-constrained LEO on-board routers. Future work will add a composable confidentiality layer, a synchronization-free anti-replay variant, and a standardization path for the Flow-Label security profile and the End.SEC behavior.

REFERENCES

- [1] S. Mahboob and L. Liu, “Revolutionizing future connectivity: A contemporary survey on ai-empowered satellite-based non-terrestrial networks in 6g,” *IEEE Commun. Surv. Tutorials*, vol. 26, no. 2, pp. 1279–1321, 2024.
- [2] O. Kodheli, E. Lagunas, N. Maturo, S. K. Sharma, S. Soderi, D. Tarchi, F. Zilli, and S. Chatzinotas, “Satellite communications in the new space era: A survey and future challenges,” *IEEE Commun. Surv. Tutorials*, vol. 23, no. 1, pp. 70–109, 2021.
- [3] 3rd Generation Partnership Project (3GPP), “Study on new radio (NR) to support non-terrestrial networks (ntn),” *3GPP Technical Report*, vol. 38.821, 2023.
- [4] M. Handley, “Delay is not an option: Low latency routing in space,” in *Proceedings of the 17th ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2018, pp. 85–91.
- [5] D. Bhattacharjee, W. Jiang, D. Kaspar, and A. Singla, “Starrynet: Empowering researchers to study starlink,” in *Proceedings of the 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2023, pp. 1319–1334.
- [6] P. Yue, J. An, J.-K. Zhang, J. Ye, G. Pan, S. Wang, P. Xiao, and L. Hanzo, “Low earth orbit satellite security and reliability: Issues, solutions, and the road ahead,” *IEEE Commun. Surv. Tutorials*, vol. 25, no. 3, pp. 1604–1652, 2023.
- [7] H. Wu, J. Hu, Z. Chen, Y. Liu, and W. Wang, “Security challenges and solutions for leo satellite networks: A comprehensive survey,” *IEEE Commun. Surv. Tutorials*, vol. 26, no. 3, pp. 1823–1865, 2024.
- [8] G. Giuliani, T. Klenze, M. Legner, D. A. Basin, A. Perrig, and A. Singla, “Internet backbones in space,” *Comput. Commun. Rev.*, vol. 50, no. 1, pp. 25–37, 2020.
- [9] B. G. Evans, “Satellite communication systems,” *IEE Proc.-Commun.*, vol. 152, no. 3, pp. 297–305, 2005.
- [10] B. Denby and B. Lucia, “Orbital edge computing: Nanosatellite constellations as a new class of computer system,” in *Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. ACM, 2020, pp. 939–954.
- [11] S. T. Kent and K. Seo, “Security architecture for the internet protocol,” *RFC*, vol. 4301, pp. 1–101, 2005.
- [12] E. Rescorla and N. Modadugu, “Datagram transport layer security version 1.2,” *RFC*, vol. 6347, pp. 1–32, 2012.
- [13] M. Caesar and J. Rexford, “BGP routing policies in ISP networks,” *IEEE Network*, vol. 19, no. 6, pp. 5–11, 2005.
- [14] Consultative Committee for Space Data Systems (CCSDS), “Space data link security protocol,” CCSDS, Tech. Rep. 355.0-B-2, Blue Book, 2022.

- [15] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment routing architecture," *RFC*, vol. 8402, pp. 1–32, 2018.
- [16] C. Filsfils, D. Dukes, S. Previdi, J. Leddy, S. Matsushima, and D. Voyer, "Ipv6 segment routing header (SRH)," *RFC*, vol. 8754, pp. 1–27, 2020.
- [17] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, S. Matsushima, and Z. Li, "Segment routing over ipv6 (srv6) network programming," *RFC*, vol. 8986, pp. 1–40, 2021.
- [18] S. Previdi, C. Filsfils, A. Abdelsalam, L. Jalil, and S. Matsushima, "BGP overlay services based on Segment Routing over IPv6 (SRv6)," *RFC*, vol. 9252, pp. 1–35, 2022.
- [19] P. L. Ventre, S. Salsano, M. Polverini, A. Cianfrani, A. Abdelsalam, C. Filsfils, P. Camarillo, and F. Clad, "Segment routing: A comprehensive survey of research activities, standardization efforts, and implementation results," *IEEE Commun. Surv. Tutorials*, vol. 23, no. 1, pp. 182–221, 2021.
- [20] A. Abdelsalam, P. Camarillo, C. Filsfils, F. Clad, S. Salsano, and P. L. Ventre, "SRv6 security considerations," in *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN)*. IEEE, 2021, pp. 88–94.
- [21] M. Tanveer, M. A. Khan, S. A. Shah, and S. Mumtaz, "SRv6-based security framework for next-generation satellite-terrestrial integrated networks," *IEEE Trans. Netw. Serv. Manag.*, vol. 21, no. 2, pp. 1897–1913, 2024.
- [22] M. S. Turan, K. McKay, D. Chang, L. E. Bassham, J. Kang, N. D. Waller, J. M. Kelsey, and D. Hong, "Status report on the final round of the NIST lightweight cryptography standardization process," National Institute of Standards and Technology, Tech. Rep. NIST IR 8454, 2023.
- [23] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, "The SIMON and SPECK lightweight block ciphers," in *Proceedings of the 52nd Annual Design Automation Conference (DAC)*. ACM, 2015, pp. 175:1–175:6.
- [24] J.-P. Aumasson and D. J. Bernstein, "Siphash: A fast short-input PRF," in *Progress in Cryptology - INDOCRYPT 2012*, ser. Lecture Notes in Computer Science, vol. 7668. Springer, 2012, pp. 489–508.
- [25] Z. Lv, Y. Li, D. Wang, and J. Chen, "Lightweight authentication for constrained on-board processors in LEO satellite constellations," in *Proceedings of the IEEE International Conference on Communications (ICC)*. IEEE, 2025, pp. 1–6.
- [26] S. Amante, B. E. Carpenter, S. Jiang, and J. Rajahalme, "Ipv6 flow label specification," *RFC*, vol. 6437, pp. 1–15, 2011.
- [27] B. E. Carpenter and S. Amante, "Using the ipv6 flow label for equal cost multipath routing and link aggregation in tunnels," *RFC*, vol. 6438, pp. 1–9, 2011.
- [28] D. Bhattacharjee and A. Singla, "Network topology design at 27,000 km/hour," in *Proceedings of the 15th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. ACM, 2019, pp. 341–354.
- [29] S. Kassing, D. Bhattacharjee, and A. Singla, "Exploring the "internet from space" with hypatia," in *Proceedings of the 20th ACM Internet Measurement Conference (IMC)*. ACM, 2023, pp. 214–229.
- [30] I. Del Portillo, B. G. Cameron, and E. F. Crawley, "A technical comparison of three leo satellite network constellations: Telesat, spacex, and amazon," *Acta Astronaut.*, vol. 159, pp. 123–135, 2019.
- [31] O. Michel and E. Keller, "Sdn in space: A novel paradigm for inter-satellite communication," in *Proceedings of the 2019 ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2019, pp. 9–15.
- [32] D. Bhattacharjee, W. Jiang, and A. Singla, "Diet sherpa: Reducing the storage overhead of ospf-based routing in satellite networks," in *Proceedings of the 2020 Workshop on Networking for Emerging Applications and Technologies (NEAT)*. ACM, 2020, pp. 1–7.
- [33] D. Bhattacharjee, S. Kassing, and A. Singla, "Towards global network super-nodes," in *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2023, pp. 180–188.
- [34] Y. Li, X. Liu, Y. Xiao, H. Zhang, Q. Qin, R. Luo, H. Cui, Z. Zhang, and Z. Li, "Internet in space for terrestrial users via cyber-physical convergence," in *Proceedings of the 21st ACM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2022, pp. 54–60.
- [35] L. Bertaux, S. Medjah, P. Berthou, S. Abdellatif, A. Hakiri, P. Gelard, F. Planchou, and M. Brévil, "Software defined networking and virtualization for broadband satellite networks," *IEEE Commun. Mag.*, vol. 53, no. 3, pp. 54–60, 2015.